Section 3: Exact Evaluation - Functional Correctness and Similarity Measurements

---

Why This Section Matters to You Practically

This section is where evaluation becomes something you build. Every AI application needs an evaluation strategy before it goes to production. The methods here - functional correctness and similarity measurements - are the backbone of any systematic evaluation pipeline.

This section answers: how do you actually measure whether your system's outputs are good? Not by eyeballing. By building evaluators.

For interviews at AI startups, "how would you evaluate this system?" is one of the most common technical questions. This section gives you the vocabulary and the tooling to answer it properly.

The Big Picture: Exact vs Subjective Evaluation

Before diving in, understand the two-categories split the book draws:

Exact evaluation produces unambiguous scores. Either the output is correct or it isn't. Either it matches the reference or it doesn't. No room for interpretation.

Subjective evaluation produces scores that depend on who or what is doing the evaluating. The same output can get different scores from different judges. AI as a judge - covered in Section 4 - falls here.

This section is entirely about exact evaluation. Two approaches: functional correctness and similarity measurements.

Approach 1: Functional Correctness

The core idea: Does the system actually do what it's supposed to do?

This is the most direct evaluation possible. You're not asking "does the output look right?" You're asking "does the output work?"

Functional correctness is the gold standard metric for any application. If your AI is supposed to book a restaurant reservation and it books the wrong restaurant, it failed - regardless of how politely it responded.

The catch: functional correctness is not always automatable. Booking a restaurant requires checking external state. Writing a compelling essay requires human judgment. But there are two categories of tasks where functional correctness can be automated cleanly.

Category 1: Code Generation

Code is the cleanest case for automated functional correctness evaluation. Why? Because code either runs or it doesn't. And if it runs, you can check whether it produces the correct output for a given input.

This is called **execution accuracy** - you execute the generated code against test cases and check the results.

The standard benchmark for this is **HumanEval** (OpenAI) and **MBPP** (Google). Each problem comes with a set of test cases. The metric used is **pass@k**:

- Generate k code samples for each problem
- A problem is considered solved if any of the k samples passes all test cases
- Final score = fraction of problems solved

So pass@1 means you generate one attempt per problem. Pass@10 means you generate ten attempts and get credit if any one of them works.

The implication: pass@k scores increase with k. A model might solve 40% of problems with one attempt but 65% with ten attempts. This tells you something real - the model knows the solution space but isn't always landing on it first try. For production, you care about pass@1. For research, pass@k gives a fuller picture of capability.

The same principle applies to **text-to-SQL** benchmarks like Spider and BIRD-SQL. Generate a SQL query, execute it against the database, check if the result matches the expected output.

Here's what a basic functional correctness evaluator looks like in code:

```python
def evaluate_code_solution(generated_code, test_cases):
    """
    Evaluates generated code against a set of test cases.
    Returns True if all test cases pass, False otherwise.
    """
    try:
        # Execute the generated code to define the function
        exec_globals = {}
        exec(generated_code, exec_globals)

        # Run each test case
        for test_input, expected_output in test_cases:
            # Get the function from executed code
            func_name = list(exec_globals.keys())[-1]
            func = exec_globals[func_name]

            actual_output = func(*test_input)
            if actual_output != expected_output:
                return False, f"Failed: input={test_input}, expected={expected_output}

        return True, "All test cases passed"
```

```python
    except Exception as e:
        return False, f"Execution error: {str(e)}"

# Example usage
generated_code = """
def gcd(num1, num2):
    while num2:
        num1, num2 = num2, num1 % num2
    return num1
"""

test_cases = [
    ((15, 20), 5),
    ((12, 8), 4),
    ((100, 75), 25)
]

passed, message = evaluate_code_solution(generated_code, test_cases)
print(f"Result: {passed}, {message}")
```

And here's pass@k implemented:

```python
def pass_at_k(problems, model_fn, k=3):
    """
    Evaluates model on a set of problems using pass@k metric.

    problems: list of (problem_description, test_cases) tuples
    model_fn: function that takes a problem and returns generated code
    k: number of attempts per problem
    """
    solved = 0

    for problem_desc, test_cases in problems:
        # Generate k attempts
        attempts = [model_fn(problem_desc) for _ in range(k)]

        # Problem is solved if any attempt passes
        problem_solved = any(
            evaluate_code_solution(code, test_cases)[0]
            for code in attempts
        )

        if problem_solved:
            solved += 1

    return solved / len(problems)
```

Category 2: Tasks With Measurable Objectives

Beyond code, any task with a quantifiable outcome can use functional correctness. A game-playing bot - measure its score. A scheduling optimizer - measure energy saved. A trading algorithm - measure returns.

The pattern is the same: define what success looks like numerically, then measure it directly.

Approach 2: Similarity Measurements Against Reference Data

When functional correctness can't be automated, the next option is to compare generated outputs against reference responses - outputs that are known to be correct or high quality.

The format is: each evaluation example is a pair of **(input, reference response)**. An input can have multiple valid reference responses. You generate an output, then measure how similar it is to the references.

Four ways to measure similarity:

Method 1: Exact Match

The generated output matches one of the reference responses exactly. Binary - match or no match.

Works well for: simple factual questions, math problems, trivia.

"What's 2 + 3?" → reference: "5" → generated: "5" ✓

"What's 2 + 3?" → reference: "5" → generated: "The answer is 5" ✗

A common variation accepts any output that *contains* the reference response. But this breaks quickly:

"What year was Anne Frank born?"

→ reference: "1929"

→ generated: "September 12, 1929"

→ contains "1929" ✓ but is factually wrong about the date

Beyond simple tasks, exact match fails. For translation, summarization, or any open-ended task, the space of valid responses is too large to enumerate.

Method 2: Lexical Similarity

Instead of requiring an exact match, measure how much the generated response *looks like* the reference - how much word or character overlap exists.

Two main approaches:

Fuzzy matching (edit distance) - count how many insertions, deletions, or substitutions it takes to convert one string to another. Fewer edits = more similar.

"bad" → "bard": 1 edit (insertion) - very similar

"bad" → "cash": 3 edits - not similar

N-gram similarity - instead of single tokens, measure overlap of sequences of tokens. A bigram is two consecutive tokens. BLEU and ROUGE use n-gram overlap.

Reference: "My cats scare the mice"

Generated A: "My cats eat the mice" → 4/5 words overlap = 80%

Generated B: "Cats and mice fight" → 3/5 words overlap = 60%

→ A is considered more similar

Common metrics: **BLEU** (used heavily in translation), **ROUGE** (used in summarization), **METEOR**, **TER**.

The critical limitation: **lexical similarity doesn't capture meaning.** "What's up?" and "How are you?" share almost no words but mean the same thing. "Let's eat, grandma" and "Let's eat grandma" share every word but mean completely different things.

Another real problem: reference sets are never exhaustive. Adept found that their model Fuyu was penalized on an image captioning benchmark because it described an image as "A nighttime view of Big Ben and the Houses of Parliament" - which was correct and more specific than the reference captions - but got a low score because none of the references mentioned Big Ben or Parliament. The model was right. The metric said it was wrong.

Method 3: Semantic Similarity

Lexical similarity measures surface overlap. Semantic similarity measures meaning.

The approach: convert both the generated response and the reference response into **embeddings** - dense vector representations that capture meaning. Then measure the distance between those vectors using cosine similarity.

You already know embeddings from Chapter 2. The key principle applies directly here: texts with similar meanings will have embeddings that are close together in vector space, regardless of the specific words used.

```python
from anthropic import Anthropic
import numpy as np

def cosine_similarity(vec_a, vec_b):
    """Compute cosine similarity between two vectors."""
    dot_product = np.dot(vec_a, vec_b)
    norm_a = np.linalg.norm(vec_a)
    norm_b = np.linalg.norm(vec_b)
    return dot_product / (norm_a * norm_b)

def semantic_similarity_eval(generated_response, reference_response, embedding_model):
    """
    Evaluate semantic similarity between generated and reference responses.
    Returns similarity score between -1 and 1.
    1 = identical meaning, 0 = unrelated, -1 = opposite meaning
    """
    # Get embeddings for both responses
    gen_embedding = embedding_model.embed(generated_response)
    ref_embedding = embedding_model.embed(reference_response)

    similarity = cosine_similarity(gen_embedding, ref_embedding)
    return similarity
```

```python
def evaluate_with_multiple_references(generated, references, embedding_model, threshold=0.85):
    """
    Evaluate against multiple reference responses.
    Pass if similarity to any reference exceeds threshold.
    """
    similarities = [
        semantic_similarity_eval(generated, ref, embedding_model)
        for ref in references
    ]

    best_similarity = max(similarities)
    passed = best_similarity >= threshold

    return {
        "passed": passed,
        "best_similarity": best_similarity,
        "all_similarities": similarities
    }
```

Common metrics for semantic similarity: **BERTScore** (uses BERT embeddings), **MoverScore**.

The limitation: semantic similarity is only as good as the embedding model. Two texts with the same meaning will score low if the embeddings are poor quality. And computing embeddings adds latency and cost to your evaluation pipeline.

One important note the book flags: **semantic similarity is technically subjective** because different embedding models can produce different scores for the same pair of texts. Given two embeddings though, cosine similarity is computed exactly. It sits in a gray zone between exact and subjective evaluation.

Section Summary

Method	What It Measures	Best For	Limitation
Functional Correctness	Does it work?	Code, measurable objectives	Hard to automate for open-ended tasks
Exact Match	Word-for-word match	Simple factual, math	Fails on any paraphrase
Lexical Similarity	Surface word overlap	Translation, summarization	Misses meaning, needs exhaustive references
Semantic Similarity	Meaning via embeddings	Open-ended responses	Quality depends on embedding model

---

Quiz - Three Questions

Q1. You're building an evaluation pipeline for a text-to-SQL feature. Users type a natural language question, the model generates a SQL query, and it runs against a database. What evaluation method would you use and why? Write the core logic of the evaluator in code.

Q2. A teammate proposes using ROUGE scores to evaluate your system's outputs. You have a case where the reference response is "The server is down due to a network timeout" and the model generates "A network timeout caused the server to become unavailable." ROUGE gives this a low score. What's wrong with using ROUGE here, and what would you use instead?

Q3. Exact match, lexical similarity, and semantic similarity each have a scenario where they're the right tool. Give me one concrete scenario for each where that specific method is clearly the best choice - and explain why the other two would fail in that scenario.